Costruire figure scientifiche con Python
Un percorso graduale con NumPy, Matplotlib e SciPy

Giuseppe Mandaglio

Indice

1 Introduzione **5**
1.1 Struttura dei file di dati 5

2 Dal grafico minimo alla figura completa **7**
2.1 Il grafico più semplice: `plot_bad.py` 7
2.2 Rappresentare punti separati: `plot_bad2.py` 8
2.3 Etichetta e legenda: `plot_bad3.py` 9
2.4 Nomi e unità degli assi: `plot_bad4.py` 10
2.5 Due serie di dati: `plot_bad5.py` 11
2.6 Lettura compatta e salvataggio: `plot_bad6.py` 12
2.7 Versione rifinita: `plottos2.py` 13
2.7.1 Perché usare `fig` e `ax` 14
2.7.2 Barre di errore con `ax.errorbar` 15
2.7.3 Etichette e caratteri 15
2.7.4 Tick principali e secondari 15
2.7.5 Cornice, legenda e layout 16

3 Fit parabolici con SciPy e NumPy **17**
3.1 Problema matematico 17
3.2 Fit con `scipy.optimize.curve_fit` 17
3.2.1 Definizione del modello 18
3.2.2 Minimizzazione e chi-quadro 18
3.2.3 Parametri restituiti e operatore `*` 19
3.2.4 Costruzione della curva continua 19
3.3 Fit con `numpy.polyfit` 19
3.3.1 Metodo dei minimi quadrati in NumPy 20
3.3.2 Fit pesato con NumPy 20
3.4 Fit con NumPy e matrice di covarianza 20
3.5 Significato della matrice di covarianza 21
3.6 Propagazione della covarianza alla curva di fit 22
3.7 Confronto tra NumPy e SciPy 23
3.8 Stampa corretta dei parametri 23

4 Buone pratiche conclusive **25**

Capitolo 1

Introduzione

Questo documento illustra, attraverso una sequenza di script di complessità crescente, come costruire una figura scientifica in Python. Si parte da un grafico essenziale e si introducono progressivamente marker, legenda, etichette degli assi, più serie di dati, salvataggio su file, barre di errore e personalizzazione grafica avanzata. L'ultimo capitolo è dedicato al fit parabolico, confrontando l'approccio specializzato di NumPy con quello più generale di SciPy.

I pacchetti utilizzati sono:

- `NumPy`, per la lettura e la manipolazione numerica dei dati;
- `Matplotlib`, per la costruzione delle figure;
- `SciPy`, per il fit di funzioni definite dall'utente.

1.1 Struttura dei file di dati

Il file `datios.txt` contiene quattro colonne:

$$x, \qquad y, \qquad \sigma_x, \qquad \sigma_y,$$

dove σ_x e σ_y rappresentano le incertezze associate alle due coordinate. Il file `Kter.txt` contiene invece tre colonne:

$$x, \qquad y_1, \qquad y_2.$$

Negli script originali il secondo file è indicato come `Kter2.dat`; nel presente documento si usa `Kter.txt`, cioè il nome del file effettivamente disponibile.

Capitolo 2

Dal grafico minimo alla figura completa

2.1 Il grafico più semplice: `plot_bad.py`

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 dati = np.loadtxt("datios.txt")
5 x = dati[:, 0]
6 y = dati[:, 1]
7 plt.plot(x, y)
8 plt.show()
```

Spiegazione delle istruzioni

`import numpy as np`
Importa NumPy e assegna al modulo l'abbreviazione convenzionale `np`.

`import matplotlib.pyplot as plt`
Importa l'interfaccia procedurale di Matplotlib con l'abbreviazione `plt`.

`np.loadtxt("datios.txt")`
Legge il file testuale e costruisce un array bidimensionale. Ogni riga del file diventa una riga dell'array e ogni colonna del file diventa una colonna dell'array.

`dati[:, 0]`
Il simbolo `:` seleziona tutte le righe, mentre `0` seleziona la prima colonna. Il risultato viene assegnato a `x`.

`dati[:, 1]`
Seleziona la seconda colonna e la assegna a `y`.

`plt.plot(x, y)`
Disegna y in funzione di x. In assenza di opzioni, Matplotlib collega i punti mediante una linea continua.

`plt.show()`
Visualizza la figura in una finestra grafica oppure nel backend utilizzato dall'ambiente di lavoro.

Questo primo script è sufficiente per verificare rapidamente l'andamento dei dati, ma la figura non contiene marker, legenda, nomi degli assi o unità di misura.

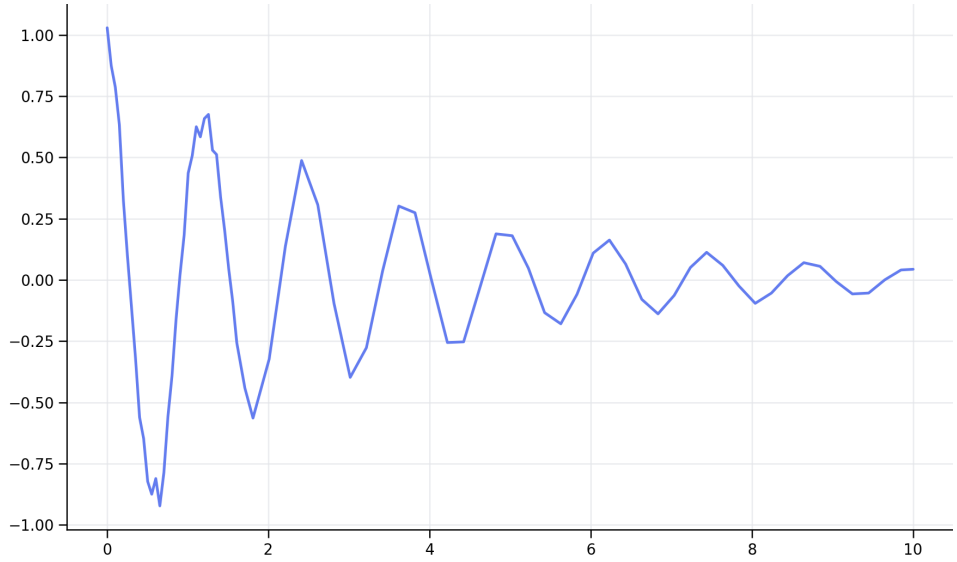


Figura 2.1: Risultato prodotto da `plot_bad.py`.

2.2 Rappresentare punti separati: `plot_bad2.py`

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 dati = np.loadtxt("datios.txt")
5 x = dati[:, 0]
6 y = dati[:, 1]
7 plt.plot(x, y, "o")
8 plt.show()
```

La sola differenza rispetto allo script precedente è la stringa di formato `"o"`. Essa richiede marker circolari. Poiché non viene specificato uno stile di linea, i punti non sono collegati. Questa rappresentazione è più adatta quando si vogliono mostrare misure discrete senza suggerire che tra due punti consecutivi sia stata osservata una linea continua.

La stringa di formato può combinare colore, marker e stile della linea. Per esempio, `"ro-"` indica cerchi rossi collegati da una linea continua. È tuttavia spesso più leggibile usare argomenti espliciti, come `marker="o"`, `color="red"` e `linestyle="-"`.

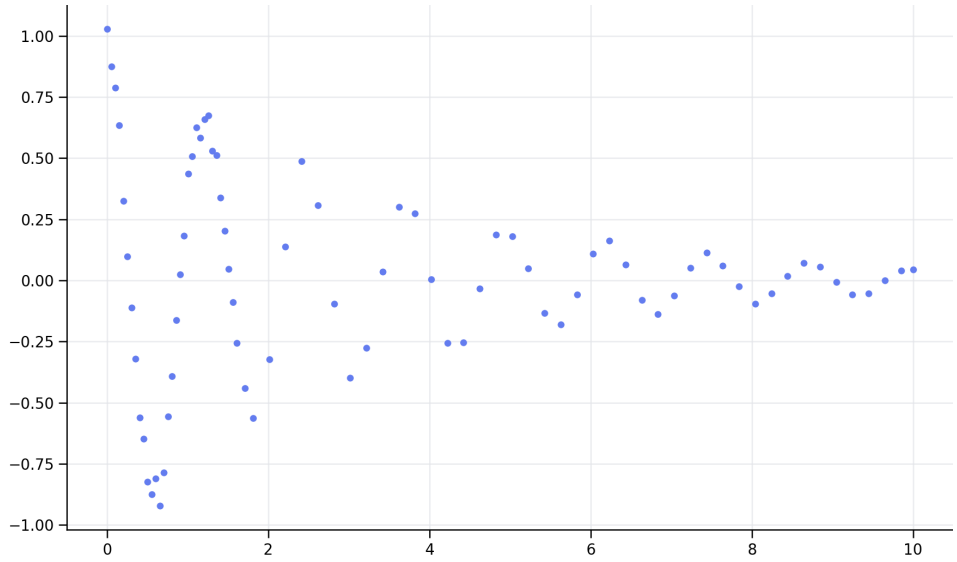


Figura 2.2: Dati rappresentati mediante marker circolari.

2.3 Etichetta e legenda: `plot_bad3.py`

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 dati = np.loadtxt("datios.txt")
5 x = dati[:, 0]
6 y = dati[:, 1]
7 plt.plot(x, y, "^", label="Misure")
8 plt.legend()
9 plt.show()
```

Il marker `"^"` produce triangoli rivolti verso l'alto. L'argomento `label="Misure"` associa un nome all'oggetto grafico creato da `plot`. La chiamata `plt.legend()` raccoglie gli oggetti che possiedono una label e costruisce la legenda.

È importante distinguere i due passaggi: `label` definisce il testo associato alla serie, mentre `legend()` rende visibile la legenda. Se si omette `legend()`, la label rimane associata all'oggetto ma non viene mostrata.

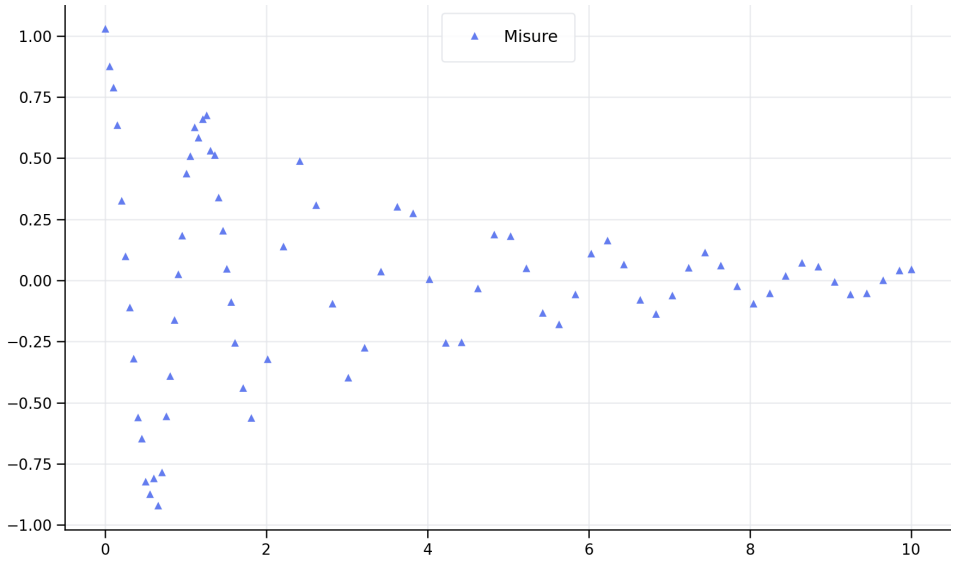


Figura 2.3: Marker triangolari e legenda.

2.4 Nomi e unità degli assi: `plot_bad4.py`

```
import numpy as np
import matplotlib.pyplot as plt

dati = np.loadtxt("datios.txt")
x = dati[:, 0]
y = dati[:, 1]
plt.plot(x, y, "^", label="Misure")
plt.xlabel("Tempo (s)")
plt.ylabel("Distanza (cm)")
plt.legend()
plt.show()
```

Le funzioni `xlabel` e `ylabel` descrivono rispettivamente la grandezza riportata sull'asse orizzontale e quella riportata sull'asse verticale. In una figura scientifica si dovrebbero indicare sia il nome della grandezza sia l'unità di misura.

Il testo riportato sugli assi non modifica i dati e non esegue conversioni di unità. Se, per esempio, i dati fossero espressi in metri ma si volessero mostrare in centimetri, sarebbe necessario trasformare numericamente l'array prima del grafico.

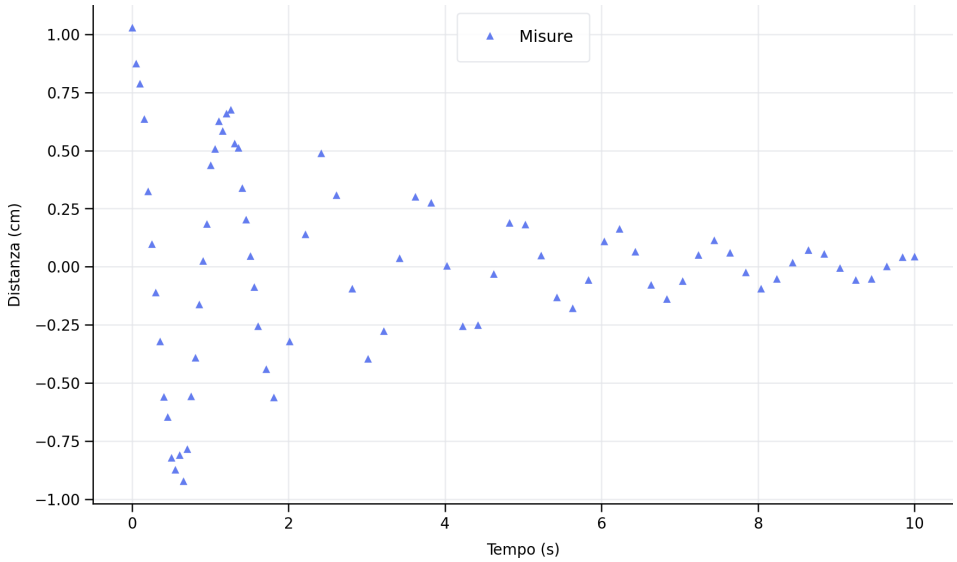


Figura 2.4: Figura completa delle etichette degli assi.

2.5 Due serie di dati: `plot_bad5.py`

```
import numpy as np
import matplotlib.pyplot as plt

dati = np.loadtxt("Kter.txt")
x = dati[:, 0]
y1 = dati[:, 1]
y2 = dati[:, 2]

plt.plot(x, y1, "^", label="Misure 1")
plt.plot(x, y2, "o", label="Misure 2")
plt.xlabel("Distanza (cm)")
plt.ylabel("Frequenza (Hz)")
plt.legend()
plt.show()
```

Questo script legge tre colonne. La prima rappresenta la variabile indipendente comune alle due misure, mentre la seconda e la terza rappresentano due serie diverse. Le due chiamate a `plot` operano sugli stessi assi e sovrappongono le serie.

Marker e label differenti permettono di distinguere i due insiemi di dati anche quando i colori non sono facilmente riconoscibili. Questa scelta migliora la leggibilità nella stampa in scala di grigi e per utenti con difficoltà nella percezione dei colori.

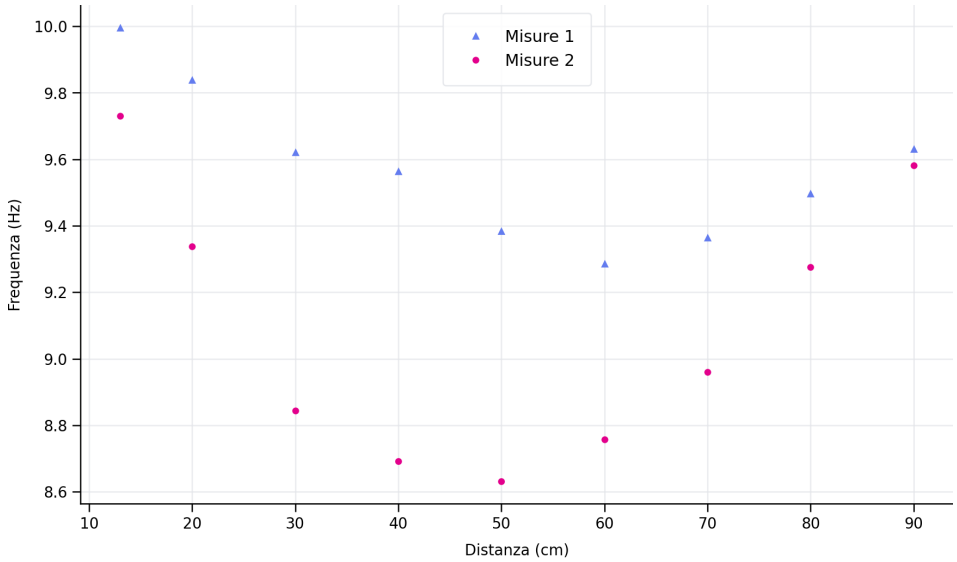


Figura 2.5: Confronto tra due serie di misure.

2.6 Lettura compatta e salvataggio: `plot_bad6.py`

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  x, y1, y2 = np.loadtxt("Kter.txt", unpack=True)
5
6  plt.plot(x, y1, "^", label="Misure 1")
7  plt.plot(x, y2, "o", label="Misure 2")
8  plt.xlabel("Distanza (cm)")
9  plt.ylabel("Frequenza (Hz)")
10 plt.legend()
11
12 plt.savefig("grafico.pdf")
13 plt.savefig("grafico.png", dpi=300)
14 plt.show()
```

L'opzione `unpack=True` traspone concettualmente il risultato della lettura: le colonne vengono restituite come array separati e possono essere assegnate direttamente a `x`, `y1` e `y2`. Questo evita la selezione manuale mediante `dati[:, i]`.

Le due chiamate a `savefig` salvano la stessa figura in formati differenti:

- il PDF conserva gli elementi grafici in forma vettoriale ed è particolarmente adatto a relazioni e pubblicazioni;
- il PNG è un formato raster; il parametro `dpi=300` richiede una risoluzione elevata, adatta alla stampa.

È opportuno salvare la figura prima di `show()`. Il comportamento di `show()` dipende dal backend e, in alcuni ambienti, la finestra può essere chiusa o la figura corrente può non essere più disponibile dopo la visualizzazione.

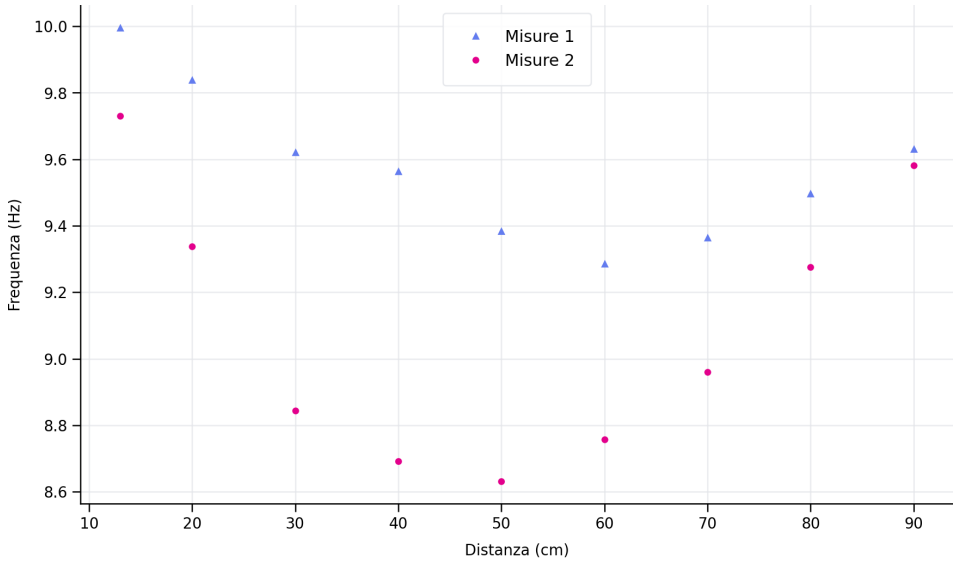


Figura 2.6: Figura a due serie, pronta per essere salvata in PDF e PNG.

2.7 Versione rifinita: plottos2.py

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import MultipleLocator, AutoMinorLocator

# Lettura dati
x, y, xerr, yerr = np.loadtxt("datios.txt", unpack=True)

# Creazione figura
fig, ax = plt.subplots(figsize=(8, 6))

# Dati con barre di errore
ax.errorbar(
    x,
    y,
    xerr=xerr,
    yerr=yerr,
    fmt="o",
    color="blue",
    ecolor="gray",
    elinewidth=1,
    capsize=3,
    markersize=5,
    label="Misura",
)

# Etichette degli assi
ax.set_xlabel(
    "Tempo (s)",
    fontsize=24,
    fontweight="normal",
    fontname="Times",
)
ax.set_ylabel(
    "Distance (cm)",
```

```
    fontsize=24,
    fontweight="normal",
    fontname="Times",
)

# Tick principali
ax.tick_params(
    axis="both",
    which="major",
    labelsize=22,
    width=1.8,
    length=7,
    color="black",
)

# Tick secondari
ax.tick_params(
    axis="both",
    which="minor",
    width=1.2,
    length=4,
    color="black",
)

# Font dei valori numerici sugli assi
for label in ax.get_xticklabels() + ax.get_yticklabels():
    label.set_fontname("Times New Roman")

# Divisioni principali e secondarie
ax.xaxis.set_major_locator(MultipleLocator(1.0))
ax.yaxis.set_major_locator(MultipleLocator(0.5))
ax.xaxis.set_minor_locator(AutoMinorLocator(5))
ax.yaxis.set_minor_locator(AutoMinorLocator(5))

# Aspetto della cornice
for spine in ax.spines.values():
    spine.set_linewidth(1.3)
    spine.set_color("black")

# Legenda
ax.legend(
    loc="best",
    prop={"family": "Times New Roman", "size": 12},
)

# Layout finale
fig.tight_layout()
plt.show()
```

2.7.1 Perché usare `fig` e `ax`

La chiamata

```
fig, ax = plt.subplots(figsize=(8, 6))
```

usa l'interfaccia orientata agli oggetti di Matplotlib. `fig` rappresenta il contenitore complessivo, cioè l'intera figura, mentre `ax` rappresenta il sistema di assi in cui vengono disegnati dati, etichette e legenda.

Questa impostazione è preferibile nei programmi articolati perché rende esplicito l'oggetto che si sta modificando. Inoltre, consente di gestire facilmente più pannelli nella stessa figura. `figsize=(8,6)` imposta larghezza e altezza in pollici; la forma $4:3$ è adatta a molte figure scientifiche.

2.7.2 Barre di errore con `ax.errorbar`

Gli argomenti `xerr=xerr` e `yerr=yerr` specificano le incertezze lungo gli assi orizzontale e verticale. Se gli array hanno la stessa lunghezza dei dati, ogni punto possiede la propria incertezza. Un singolo valore scalare, invece, assegnerebbe la stessa incertezza a tutti i punti.

Le principali opzioni grafiche sono:

`fmt="o"`
: usa cerchi come marker e non aggiunge una linea tra i punti;

`color="blue"`
: imposta il colore dei marker;

`ecolor="gray"`
: imposta separatamente il colore delle barre di errore;

`elinewidth=1`
: stabilisce lo spessore delle barre;

`capsize=3`
: aggiunge piccoli segmenti terminali lunghi tre punti tipografici;

`markersize=5`
: imposta la dimensione dei marker;

`label="Misura"`
: assegna alla serie il testo usato dalla legenda.

2.7.3 Etichette e caratteri

I metodi `ax.set_xlabel` e `ax.set_ylabel` modificano le etichette del singolo oggetto `ax`. `fontsize=24` imposta la dimensione, `fontweight="normal"` il peso e `fontname` la famiglia tipografica.

Se il font richiesto non è installato, Matplotlib usa un font sostitutivo. Per ottenere un aspetto uniforme è opportuno usare la stessa famiglia per etichette, numeri degli assi e legenda. Nel codice è inoltre consigliabile uniformare la lingua, sostituendo eventualmente `Distance` con `Distanza`.

2.7.4 Tick principali e secondari

`tick_params` controlla l'aspetto dei tick e delle relative etichette:

- `axis="both"` applica le modifiche a entrambi gli assi;
- `which="major"` o `which="minor"` seleziona i tick principali o secondari;
- `labelsize` modifica la dimensione dei valori numerici;
- `width` imposta lo spessore dei tick;
- `length` ne imposta la lunghezza;
- `color` ne imposta il colore.

Le istruzioni con `MultipleLocator` stabiliscono il passo delle divisioni principali. Sull'asse x un tick principale compare ogni 1.0, mentre sull'asse y compare ogni 0.5. `AutoMinorLocator(5)` suddivide ciascun intervallo tra due tick principali in cinque parti uguali; di conseguenza, tra due tick principali compaiono quattro tick interni.

2.7.5 Cornice, legenda e layout

Gli elementi di `ax.spines` rappresentano i quattro lati della cornice. Il ciclo applica lo stesso spessore e lo stesso colore a tutti i lati.

In `ax.legend`, `loc="best"` lascia a Matplotlib la scelta della posizione che tende a sovrapporsi meno ai dati. L'argomento `prop` controlla famiglia e dimensione del carattere della legenda. Infine, `fig.tight_layout()` ridistribuisce automaticamente gli spazi per ridurre il rischio che etichette o numeri vengano tagliati ai bordi.

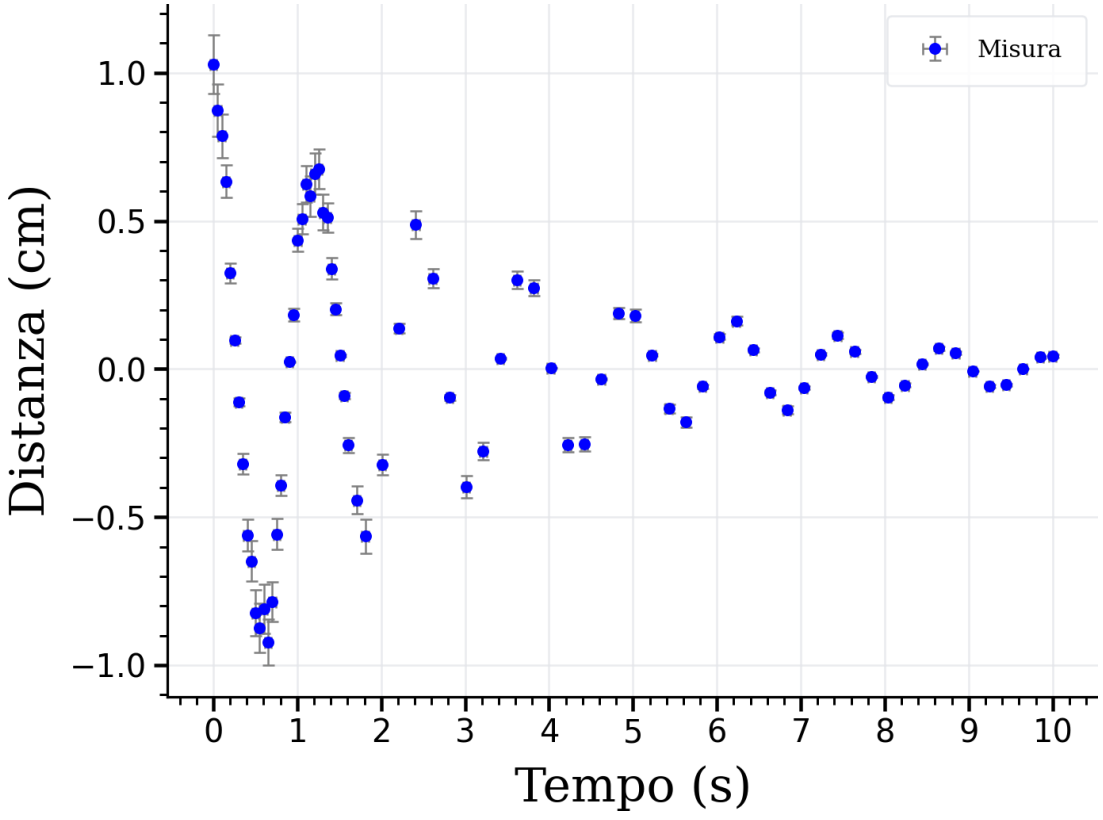


Figura 2.7: Versione rifinita con barre di errore, tick principali e secondari e stile personalizzato.

Capitolo 3

Fit parabolici con SciPy e NumPy

3.1 Problema matematico

Gli script assumono che i dati possano essere descritti dal modello

$$f(x; a, b, c) = ax^2 + bx + c.$$

Per ogni punto sperimentale si definisce il residuo

$$r_i = y_i - f(x_i; a, b, c).$$

Nel caso non pesato, il fit determina i parametri che minimizzano

$$S(a, b, c) = \sum_{i=1}^{N} r_i^2.$$

Se sono note le incertezze σ_i sulle ordinate, si minimizza invece il chi-quadro

$$\chi^2(a, b, c) = \sum_{i=1}^{N} \left[\frac{y_i - f(x_i; a, b, c)}{\sigma_i} \right]^2 .$$

Quando tutte le incertezze sono uguali, minimizzare il chi-quadro equivale, a meno di un fattore costante, a minimizzare la somma dei quadrati dei residui.

3.2 Fit con `scipy.optimize.curve_fit`

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Funzione di fit
def parabola(x, a, b, c):
    return a*x**2 + b*x + c

# Lettura dati
x, y1, y2 = np.loadtxt("Kter.txt", unpack=True)

# Fit
popt1, pcov1 = curve_fit(parabola, x, y1)
popt2, pcov2 = curve_fit(parabola, x, y2)

# Errori sui parametri
```

```
perr1 = np.sqrt(np.diag(pcov1))
perr2 = np.sqrt(np.diag(pcov2))

# Griglia fitta per il disegno
xfit = np.linspace(np.min(x), np.max(x), 500)

plt.plot(x, y1, "^", label="Misure 1")
plt.plot(x, y2, "o", label="Misure 2")
plt.plot(xfit, parabola(xfit, *popt1),
         "r-", lw=2, label="Fit 1")
plt.plot(xfit, parabola(xfit, *popt2),
         "b-", lw=2, label="Fit 2")
plt.xlabel("Distanza (cm)")
plt.ylabel("Frequenza (Hz)")
plt.legend()
plt.savefig("grafico.pdf")
plt.savefig("grafico.png", dpi=300)
plt.show()
```

3.2.1 Definizione del modello

La funzione `parabola` riceve come primo argomento la variabile indipendente e, successivamente, i parametri da determinare. Questa struttura è richiesta da `curve_fit`. Il vantaggio è che il modello non deve essere necessariamente un polinomio: può essere una funzione esponenziale, gaussiana, trigonometrica oppure una combinazione più complessa.

3.2.2 Minimizzazione e chi-quadro

`curve_fit` esegue un fit ai minimi quadrati non lineari. Se `sigma` non viene specificato, il programma assume pesi uguali, equivalenti a un array di incertezze tutte unitarie, e minimizza la somma dei quadrati dei residui. Se si fornisce un array monodimensionale `sigma`, la funzione minimizzata è

$$\chi^2 = \sum_i \left(\frac{r_i}{\sigma_i}\right)^2 .$$

Pertanto, è corretto descrivere `curve_fit` come una procedura di minimizzazione del chi-quadro quando le incertezze sono esplicitamente fornite; senza `sigma`, si tratta del caso non pesato dello stesso schema.

Per un fit pesato si scrive:

```
popt, pcov = curve_fit(
    parabola,
    x,
    y,
    sigma=yerr,
    absolute_sigma=True,
)
```

Con `absolute_sigma=True`, le incertezze fornite sono interpretate in senso assoluto e la matrice di covarianza dei parametri mantiene questa normalizzazione. Con il valore predefinito `False`, conta inizialmente la scala relativa delle incertezze e la covarianza viene riscalata in relazione al chi-quadro ridotto ottenuto.

Per problemi senza vincoli, `curve_fit` usa normalmente l'algoritmo di Levenberg–Marquardt. In presenza di limiti sui parametri passa a metodi compatibili con i vincoli, come il Trust Region Reflective. Nel caso della parabola il modello è lineare nei parametri a, b e c, anche se `curve_fit` viene utilizzato attraverso la sua interfaccia generale per fit non lineari.

3.2.3 Parametri restituiti e operatore *

`popt1` contiene i parametri ottimali nell'ordine con cui compaiono nella funzione:

$$\texttt{popt1} = [a, b, c].$$

L'espressione `parabola(xfit, *popt1)` usa l'operatore di spacchettamento `*`: i tre elementi dell'array sono passati alla funzione come tre argomenti distinti.

3.2.4 Costruzione della curva continua

`np.linspace` genera 500 punti equidistanti tra il minimo e il massimo valore di x. Il fit viene valutato su questa griglia più fitta, non soltanto nei punti sperimentali. In questo modo la curva appare regolare. I dati restano rappresentati con marker, mentre il modello è mostrato con una linea continua di spessore `lw=2`.

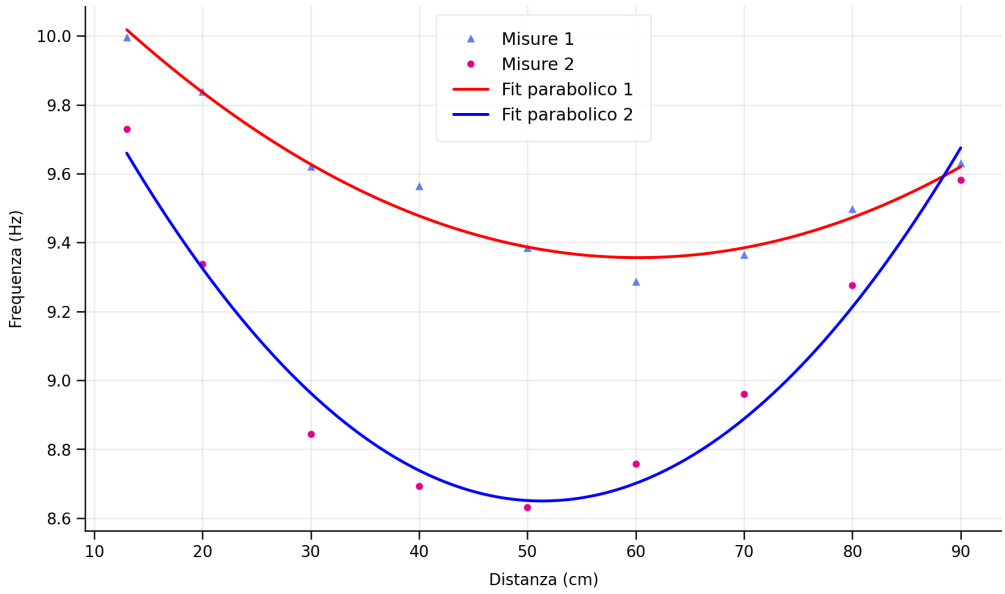


Figura 3.1: Fit delle due serie mediante `scipy.optimize.curve_fit`.

3.3 Fit con `numpy.polyfit`

```
import numpy as np
import matplotlib.pyplot as plt

x, y1, y2 = np.loadtxt("Kter.txt", unpack=True)

# Fit quadratici
p1 = np.polyfit(x, y1, 2)
p2 = np.polyfit(x, y2, 2)

# Curva liscia per il disegno
xfit = np.linspace(x.min(), x.max(), 500)
yfit1 = np.polyval(p1, xfit)
yfit2 = np.polyval(p2, xfit)

plt.plot(x, y1, "^", label="Misure 1")
plt.plot(x, y2, "o", label="Misure 2")
plt.plot(xfit, yfit1, "r-", lw=2,
```

```
         label="Fit parabolico 1")
plt.plot(xfit, yfit2, "b-", lw=2,
         label="Fit parabolico 2")
plt.xlabel("Distanza (cm)")
plt.ylabel("Frequenza (Hz)")
plt.legend()
plt.savefig("grafico.pdf")
plt.savefig("grafico.png", dpi=300)
plt.show()
```

3.3.1 Metodo dei minimi quadrati in NumPy

`np.polyfit(x,y,2)` costruisce un polinomio di grado 2 e restituisce i coefficienti che minimizzano la somma dei quadrati dei residui. Il problema viene scritto nella forma matriciale

$$\mathbf{A}\mathbf{p} \simeq \mathbf{y},$$

dove

$$\mathbf{A} = \begin{pmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ \vdots & \vdots & \vdots \\ x_N^2 & x_N & 1 \end{pmatrix}, \qquad \mathbf{p} = \begin{pmatrix} a \\ b \\ c \end{pmatrix}.$$

La matrice $\mathbf{A}$ è una matrice di Vandermonde. NumPy risolve il problema lineare ai minimi quadrati usando una decomposizione numerica collegata alla SVD, evitando di calcolare esplicitamente l'inversa di $\mathbf{A}^T\mathbf{A}$.

Il valore `2` è il grado del polinomio. I coefficienti sono ordinati dalla potenza più alta alla più bassa:

$$\texttt{p1[0]} = a, \qquad \texttt{p1[1]} = b, \qquad \texttt{p1[2]} = c.$$

`np.polyval(p1,xfit)` valuta il polinomio nei punti della griglia `xfit`.

3.3.2 Fit pesato con NumPy

Se sono disponibili le incertezze σ_i sulle ordinate, si può usare

```
p = np.polyfit(x, y, 2, w=1/yerr)
```

I pesi di `polyfit` moltiplicano i residui non elevati al quadrato. Per ottenere il consueto chi-quadro, il peso corretto è quindi $w_i = 1/\sigma_i$, non $1/\sigma_i^2$.

3.4 Fit con NumPy e matrice di covarianza

```
import numpy as np
import matplotlib.pyplot as plt

x, y1, y2 = np.loadtxt("Kter.txt", unpack=True)

# Fit quadratici con matrice di covarianza
p1, cov1 = np.polyfit(x, y1, 2, cov=True)
p2, cov2 = np.polyfit(x, y2, 2, cov=True)

# Incertezze standard sui parametri
err1 = np.sqrt(np.diag(cov1))
err2 = np.sqrt(np.diag(cov2))

```

```
xfit = np.linspace(x.min(), x.max(), 500)
yfit1 = np.polyval(p1, xfit)
yfit2 = np.polyval(p2, xfit)

plt.plot(x, y1, "^", label="Misure 1")
plt.plot(x, y2, "o", label="Misure 2")
plt.plot(xfit, yfit1, "r-", lw=2,
         label="Fit parabolico 1")
plt.plot(xfit, yfit2, "b-", lw=2,
         label="Fit parabolico 2")
plt.xlabel("Distanza (cm)")
plt.ylabel("Frequenza (Hz)")
plt.legend()

print("\nFit 1")
print(f"a = {p1[0]:.5g} +/- {err1[0]:.2g}")
print(f"b = {p1[1]:.5g} +/- {err1[1]:.2g}")
print(f"c = {p1[2]:.5g} +/- {err1[2]:.2g}")

print("\nFit 2")
print(f"a = {p2[0]:.5g} +/- {err2[0]:.2g}")
print(f"b = {p2[1]:.5g} +/- {err2[1]:.2g}")
print(f"c = {p2[2]:.5g} +/- {err2[2]:.2g}")

plt.savefig("grafico.pdf")
plt.savefig("grafico.png", dpi=300)
plt.show()
```

3.5 Significato della matrice di covarianza

La matrice di covarianza dei parametri è, nel caso di tre parametri,

$$\mathbf{C} = \begin{pmatrix} \mathrm{Var}(a) & \mathrm{Cov}(a,b) & \mathrm{Cov}(a,c) \\ \mathrm{Cov}(b,a) & \mathrm{Var}(b) & \mathrm{Cov}(b,c) \\ \mathrm{Cov}(c,a) & \mathrm{Cov}(c,b) & \mathrm{Var}(c) \end{pmatrix}.$$

Essa descrive sia le incertezze individuali dei parametri sia le loro correlazioni statistiche.

Gli elementi diagonali sono le varianze stimate:

$$C_{aa} = \sigma_a^2, \qquad C_{bb} = \sigma_b^2, \qquad C_{cc} = \sigma_c^2.$$

Per questo motivo le incertezze standard si ricavano mediante

```
err = np.sqrt(np.diag(cov))
```

Gli elementi fuori diagonale sono covarianze. Una covarianza diversa da zero indica che le variazioni ammissibili di due parametri non sono indipendenti. In un fit parabolico, per esempio, una variazione del coefficiente quadratico può essere parzialmente compensata da una variazione del coefficiente lineare o del termine costante.

La matrice è utile per:

- riportare correttamente le incertezze sui parametri;
- valutare le correlazioni tra i parametri;
- propagare le incertezze a una quantità dipendente dai parametri;

- costruire bande di incertezza della funzione di fit;
- riconoscere fit mal condizionati, nei quali alcuni parametri risultano fortemente correlati.

La correlazione normalizzata tra due parametri p_i e p_j è

$$\rho_{ij} = \frac{C_{ij}}{\sqrt{C_{ii}C_{jj}}},$$

con valori compresi tra -1 e $+1$. Valori in modulo vicini a uno indicano una forte correlazione o anticorrelazione.

In `np.polyfit(...,cov=True)`, la covarianza è normalmente scalata in modo che il chi-quadro ridotto sia unitario. Se si forniscono pesi derivati da incertezze assolute affidabili, si può richiedere

```
p, cov = np.polyfit(
    x, y, 2,
    w=1/yerr,
    cov="unscaled",
)
```

In `curve_fit`, il ruolo analogo è svolto dall'opzione `absolute_sigma=True`.

3.6 Propagazione della covarianza alla curva di fit

Il valore della parabola in un punto x può essere scritto come

$$f(x) = \mathbf{v}^T\mathbf{p}, \qquad \mathbf{v} = \begin{pmatrix} x^2 \\ x \\ 1 \end{pmatrix}.$$

La varianza propagata della curva è

$$\sigma_f^2(x) = \mathbf{v}^T\mathbf{C}\mathbf{v}.$$

In Python si può calcolare una banda di incertezza con:

```
V = np.column_stack((xfit**2, xfit, np.ones_like(xfit)))
var_fit = np.einsum("ij,jk,ik->i", V, cov1, V)
sigma_fit = np.sqrt(var_fit)

plt.fill_between(
    xfit,
    yfit1 - sigma_fit,
    yfit1 + sigma_fit,
    alpha=0.25,
    label="Incertezza del fit",
)
```

Questa banda rappresenta l'incertezza associata ai parametri stimati e alle loro covarianze. Non coincide necessariamente con un intervallo di previsione per una nuova misura, che dovrebbe includere anche la dispersione sperimentale dei singoli dati.

3.7 Confronto tra NumPy e SciPy

Aspetto	`numpy.polyfit`	`scipy.optimize.curve_fit`
Tipo di modello	Polinomio di grado assegnato	Funzione Python generica
Problema numerico	Minimi quadrati lineari nei coefficienti	Minimi quadrati non lineari in forma generale
Funzione obiettivo	Somma dei quadrati, eventualmente pesata	Chi-quadro con `sigma`; caso non pesato se `sigma=None`
Valori iniziali	Non richiesti	Utili o necessari per modelli non lineari complessi
Risultato	Coefficienti polinomiali	Parametri nell'ordine della funzione
Covarianza	Con `cov=True`	Restituita come `pcov`
Pesi	`w=1/sigma`	`sigma=sigma`
Uso tipico	Regressioni polinomiali	Modelli generici, anche non lineari

È importante non contrapporre rigidamente "minimi quadrati" e "chi-quadro": il chi-quadro è una somma di residui quadratici normalizzati dalle incertezze. NumPy e SciPy appartengono quindi alla stessa famiglia concettuale. La differenza principale è che `polyfit` sfrutta la linearità del polinomio rispetto ai coefficienti e risolve direttamente un problema lineare, mentre `curve_fit` usa un algoritmo iterativo generale per minimizzare la funzione obiettivo.

Nel caso della parabola e in assenza di pesi, i due metodi devono fornire risultati compatibili entro la precisione numerica. Le differenze diventano sostanziali quando il modello non è lineare nei parametri, quando si introducono vincoli oppure quando sono necessari valori iniziali.

3.8 Stampa corretta dei parametri

La sintassi

```
1 print(f"a = {p1[0]:.5g} +/- {err1[0]:.2g}")
```

usa una f-string. L'espressione `p1[0]` seleziona il parametro, mentre `:.5g` richiede cinque cifre significative. Analogamente, `:.2g` richiede due cifre significative per l'incertezza.

Non devono comparire backslash all'interno delle parentesi quadre della f-string. Forme come `p1\[0\]` producono un errore sintattico. Inoltre, il formato deve essere applicato all'elemento dell'array e non all'array stesso.

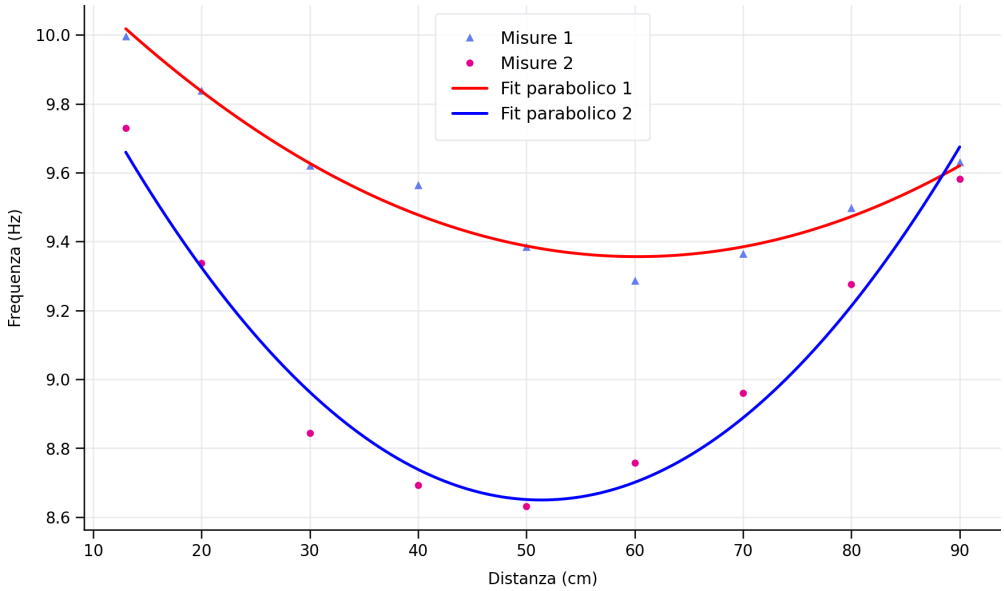


Figura 3.2: Fit parabolico ottenuto con `numpy.polyfit` e valutato con `numpy.polyval`.

Capitolo 4

Buone pratiche conclusive

- mantenere coerenti i nomi dei file negli script e nella cartella di lavoro;
- indicare sempre grandezze e unità sugli assi;
- distinguere serie diverse mediante marker oltre che mediante colori;
- usare l'interfaccia `fig, ax` per figure complesse o con più pannelli;
- rappresentare le incertezze sperimentali quando sono disponibili;
- salvare la figura prima di chiamare `show()`;
- usare PDF per la grafica vettoriale e PNG ad alta risoluzione per la grafica raster;
- usare un fit pesato quando le incertezze dei punti non sono uguali;
- riportare parametri, incertezze, covarianze e un indicatore della qualità del fit;
- controllare i residui, perché un valore accettabile del chi-quadro non sostituisce l'ispezione dell'andamento sistematico degli scarti.